

Data Analysis for a Retail Company

Project Overview:

XYZ Retail Company, a leading retailer in the industry, is looking to enhance its data analysis capabilities. To do this, they are developing a suite of Python-based tools to analyze their business data effectively. As a junior data scientist, your task is to create and implement various Python codes to solve basic data science problems related to their operations.

Task 1: Sales Data Summary

Objective: XYZ Retail wants to automate the process of calculating basic sales metrics.

- Assign the number of units sold in two categories, `Category A` and `Category B`, to variables.
- Calculate the total units sold, the difference between the categories, and the ratio of units sold.
- Print these results clearly on the management team.

Sample Input -

Category A: 150 units

Category B: 120 units

Sample Output -

- Sales Data Summary:
- Total Units Sold: 270
- Difference Between Categories: 30
- Ratio of Category A to Category B: 1.25

Task 2: Customer Age Data

Objective: Understanding the age distribution of customers is crucial for marketing strategies.

- Store a customer's name and age.
- Convert the age into a string and create a personalized marketing message like "Dear John Doe, at 30, you're eligible for our premium loyalty program."
- Print the message for use in email campaigns.

Sample Input -

Customer Name: John Doe Customer Age: 30

Sample Output -

Dear John Doe, at 30, you're eligible for our premium loyalty program.

Task 3: Product List Management

Objective: Efficient management of the product list is essential for inventory control.

- Given a list of product prices, extract the highest and lowest prices.
- Create a new list with the mid-range products.
- Add a new premium product price to the list and print the updated list for the inventory team.

Sample Input

```
product_prices = [50, 120, 30, 200, 90, 150, 180]
premium_product_price = 250
```

Sample Output

- Highest Price: 200
- Lowest Price: 30
- Mid-Range Products: [50, 120, 90, 150, 180]
- Updated Product List with Premium Price: [50, 120, 30, 200, 90, 150, 180, 250]

Task 4: Inventory Lookup

Objective: Quick access to product details is important for customer service representatives.

- Create a dictionary storing key information about a product (e.g., `product\_name`, `SKU`, `price`, `category`).
- Print the product name and SKU when queried by a customer service representative.

Sample Input

```
product_info = { "product_name": "Wireless Mouse", "SKU": "WM-12345", "price": 25.99, "category": "Electronics" }
```

Sample Output

Product Name: Wireless Mouse

Product SKU: WM-12345

Task 5: Stock Level Alert System

Objective: Ensuring that stock levels are maintained is critical to avoid stockouts.

- Write the code that takes the stock level as input.
- If stock is below a certain threshold, print a "Reorder Now" alert. If stock is above the threshold, print "Stock is sufficient."

Sample Input

Stock Level: 10

Threshold: 20

Sample Output

Enter stock level: 10
Reorder Now

Enter stock level: 25
Stock is sufficient

OR

Task 6: Sales Report Formatting

Objective: Formatting the sales data for management reports is crucial.

- Given a list of products sold, print each product name in uppercase for better visibility in reports.
- Implement both a `for` loop and a `while` loop for this task to ensure code flexibility.

Sample Input

```
products_sold = ["laptop", "mouse", "keyboard", "monitor", "printer"]
```

Sample Output

WIRELESS MOUSE
KEYBOARD
HEADPHONES
MONITOR
LAPTOP

Task 7: Area Calculation for Store Layout

Objective: Accurate area calculations are needed to plan new store layouts.

- Create a function that calculates the area of a section of the store based on length and width.
- Use this function to calculate and print the area of several store sections.

Sample Input

```
def main():
    length1, width1 = 20, 15
    print(f"The area of section 1 is {calculate_area(length1, width1)} square meters.")
```

Sample Output

The area of section 1 is 300 square meters.

Task 8: Customer Feedback Analysis

Objective: Analyzing customer feedback is vital to improving service.

- Write the code to count the number of vowels in a customer feedback message.
- Also, reverse the feedback message for a unique data presentation in reports.

Sample Input

"I loved the fast and friendly service!"

Sample Output

Customer Feedback Analysis:

Original Feedback: The service was excellent and very quick!

Number of Vowels: 13

Reversed Feedback: !kciuq yrev dna tnellecxe saw ecivres ehT

Task 9: Price Filtering Tool

Objective: Filtering product prices helps in creating targeted discounts.

- Use list to filter out products priced below a certain threshold from the product list.
- Print the list of eligible products for a discount campaign.

Sample Input

```
product_prices = [150, 85, 300, 120, 45, 200]
Discount Threshold: 100
```

Sample Output

Products eligible for the discount campaign: [85, 45]

Task 10: Daily Sales Average

Objective: Calculate the average daily sales for the past week.

- Given a list of sales figures for the last 7 days, calculate the average sales.
- Print the average sales to help the finance team understand the weekly performance.

Sample Input

```
daily_sales = [1200, 1500, 1100, 1800, 1700, 1600, 1400]
```

Sample Output

Average Daily Sales for the Past Week: \$1471.43

Task 11: Customer Segmentation

Objective: Categorize customers based on their total spending.

- Create a list of customer spending amounts.
- Use a loop to categorize customers as "Low", "Medium", or "High" spenders based on their spending amount.
- Print the categorized results to assist in targeted marketing.

Sample Input

Customer Categorization Criteria:

- Low: Spending < \$500
- Medium: \$500 <= Spending < \$1500
- High: Spending >= \$1500

Customer Spendings: [200, 800, 1500, 3000, 450, 1200]

Sample Output

Customer Spending Categories:

Customer 1: Spending = \$200 -> Category: Low
 Customer 2: Spending = \$800 -> Category: Medium
 Customer 3: Spending = \$1500 -> Category: High
 Customer 4: Spending = \$3000 -> Category: High
 Customer 5: Spending = \$450 -> Category: Low
 Customer 6: Spending = \$1200 -> Category: Medium

Task 12: Discount Calculation

Objective: Automate the calculation of discounts for a promotional campaign.

- Write a code that calculates the final price after applying a discount percentage to a product's original price.
- Test this function on a list of products with different discounts and print the final prices.

Sample Input

```
products = [ {"name": "Product A", "original_price": 100, "discount_percentage": 10},
{"name": "Product B", "original_price": 250, "discount_percentage": 20}, {"name":
"Product C", "original_price": 75, "discount_percentage": 15}, {"name": "Product D",
"original_price": 150, "discount_percentage": 5} ]
```

Sample Output

Final Prices After Discounts:

Product 1: Original Price = \$100, Discount = 10%, Final Price = \$90.00

Product 2: Original Price = \$250, Discount = 20%, Final Price = \$200.00

Product 3: Original Price = \$75, Discount = 15%, Final Price = \$63.75

Product 4: Original Price = \$150, Discount = 5%, Final Price = \$142.50

Task 13: Customer Feedback Sentiment Analysis

Objective: Basic sentiment analysis of customer feedback.

- Write the Python code that checks if certain positive or negative words (e.g., "good", "bad", "happy", "disappointed") are present in customer feedback.
- Print "Positive" or "Negative" based on the words found in the feedback.

Sample Input

Customer Feedback: "I am very happy with the service. It was a good experience!"

Positive Words: ["good", "happy", "excellent", "great"]

Negative Words: ["bad", "disappointed", "poor", "terrible"]

Sample Output

Customer Feedback Sentiment: Positive

Task 14: Employee Salary Increment Calculator

Objective: Calculate the salary increment for employees based on their performance rating.

- Create a dictionary that stores employee names and their performance ratings.
- Write the code that applies a different increment percentage based on the rating.
- Print the updated salary for each employee.

Sample Input

```
employees = { "Alice": {"current_salary": 50000, "rating": "Excellent"}, "Bob":
{"current_salary": 40000, "rating": "Good"}, "Charlie": {"current_salary": 45000,
"rating": "Average"}, "David": {"current_salary": 35000, "rating": "Poor"} }
```

```
increments = { "Excellent": 20, "Good": 15, "Average": 10, "Poor": 5 }
```

Sample Output

Updated Salaries After Increment:

Alice: Current Salary = \$50000, Rating = Excellent, Updated Salary = \$60000.00

Bob: Current Salary = \$40000, Rating = Good, Updated Salary = \$46000.00

Charlie: Current Salary = \$45000, Rating = Average, Updated Salary = \$49500.00

David: Current Salary = \$35000, Rating = Poor, Updated Salary = \$36750.00

Task 15: Stock Replenishment Planning

Objective: Determine which products need replenishment based on sales data.

- Given a list of products and their current stock levels, compare these against a predefined threshold.
- Print a list of products that need to be reordered to maintain adequate stock levels.

Sample Input

```
products = [ {"product_name": "Product A", "stock": 50}, {"product_name": "Product B",
"stock": 150}, {"product_name": "Product C", "stock": 30}, {"product_name": "Product
D", "stock": 75}, {"product_name": "Product E", "stock": 20} ]
threshold = 40
```

Sample Output

Products that need replenishment:

Product C

Product E

Project Submission Guidelines:

- Create a Python notebook file and have all the questions answered.
- Ensure that the scripts are well-commented and follow coding best practices.
- Submit all scripts in a single compressed folder named `XYZ\_Retail\_Project.zip`.

Evaluation Criteria:

- Correctness and functionality of the scripts.
- Code readability and documentation.
- Relevance and accuracy of the output generated by the scripts.

Submission Instructions:

To submit your assignment, please follow these guidelines:

- Ensure that your assignment is fully completed.
- Push your assignment to a GitHub repository.
- Share the repository link by including it in a text, Word, or PDF file.
- Submit the file/text containing the repository link via Vlearn.